

manually bypassing SSL pinning

does these protection still
matters?



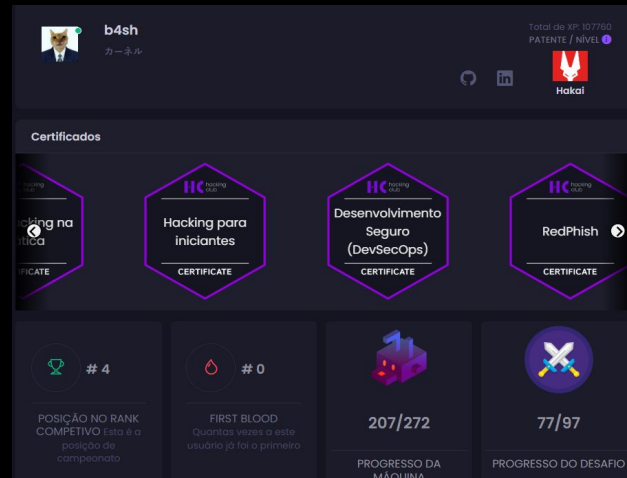
@gustavo/b4sh0xf

> <CONTEÚDO>

0x01: whoami?



- estudante de engenharia da computação no CIn-UFPE.
- estudante de segurança ofensiva (web/mobile)
- aluno/ctf player @hacking club



> <CONTEÚDO>

0x02: resumo da talk

- iremos começar compreendendo o que é o SSL pinning e terminaremos mostrando na prática como se faz o bypass dessa proteção.
- tópicos:
 - o que é o ssl pinning?
 - por quê ele é implementado?
 - ainda faz sentido implementá-lo?
 - overview sobre a linguagem SMALI
 - hands-on: bypassando manualmente o ssl pinning de uma aplicação

> <CONTEÚDO>

0x03: o que é o SSL pinning?

- técnica que consiste em registrar no aplicativo um certificado ssl ou uma chave pública (mais comum) para fazer verificar a comunicação do aplicativo com a sua api.
- o aplicativo faz a requisição. a api devolve com a chave pública do seu certificado ssl e então é feita a comparação com aquela que foi pinnada no aplicativo.

> <CONTEÚDO>

entendeu ou quer que desenhe?

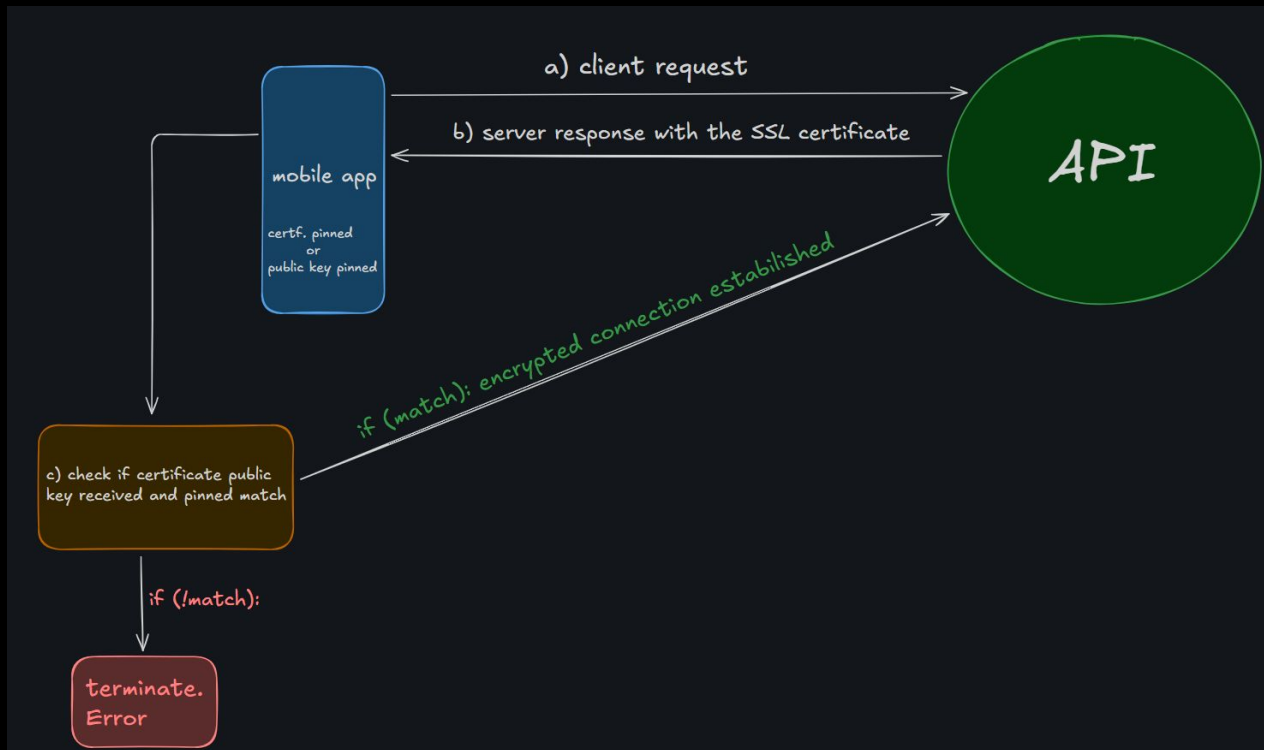


figura 1: esquema visual

> <CONTEÚDO>

0x04: por quê ele é implementado?

- essa proteção deixa o aplicativo ~~seguro~~ seguro contra a utilização de proxies para ataques MITM, evitando com que as requests feitas pelo app sejam interceptadas e/ou modificadas por web proxies.



> <CONTEÚDO>

0x05: ainda faz sentido implementá-lo?

- atualmente, essa proteção é vista como desnecessária, e tem o seu uso desencorajado por empresas como Google e Cloudflare.
- nos últimos anos, o tempo de duração dos certificados CA tem diminuído, além da rotação constante de certificados intermediários. isso implica em atualizações obrigatórias
- 100% bypassável!

> <CONTEÚDO>

vencimento de certificados SSL

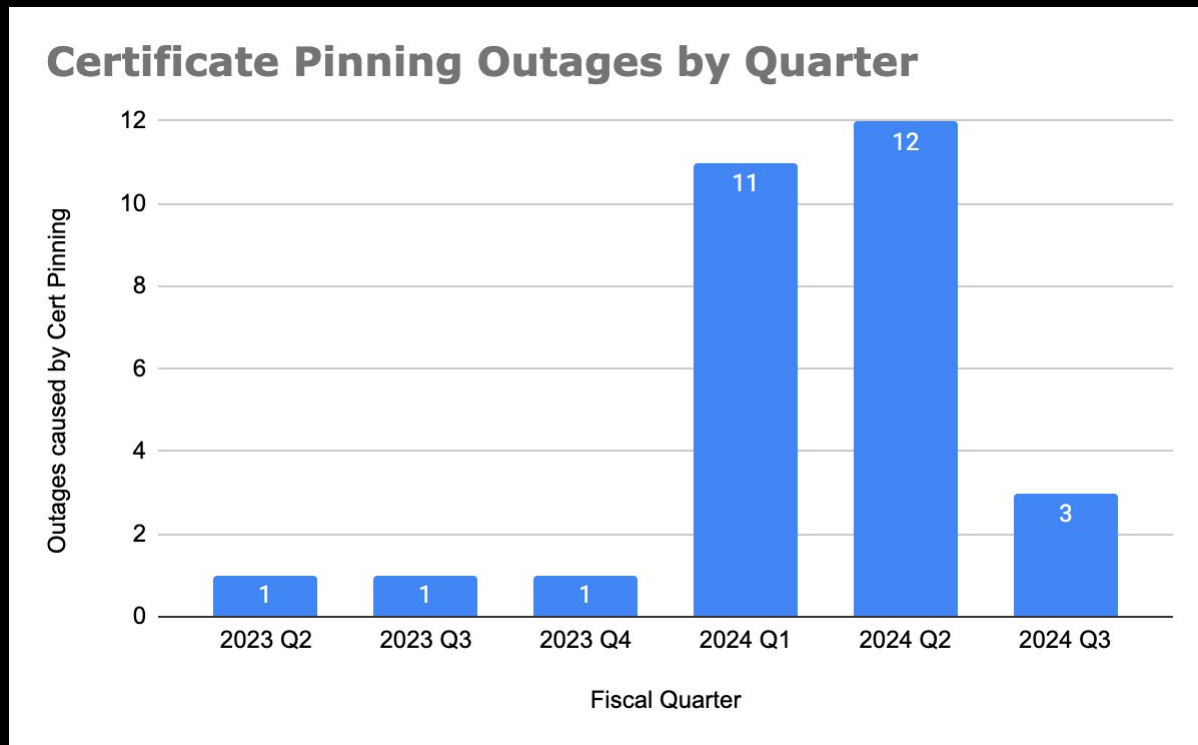


figura 2. interrupções causadas por vencimento de certificados em 2023/2024

> <CONTEÚDO>

0x06: sobre o SMALI

- linguagem semelhante ao assembly. fase intermediária entre o código de alto nível (java/kotlin/flutter) e o DEX bytecode, que é o que de fato é executado pelo ART e roda o aplicativo.
- em um cenário de ofuscação de código, para fazer uma engenharia reversa eficiente, é necessário entender o código smali do aplicativo. modificações no aplicativo também são feitas diretamente no código smali.

> <CONTEÚDO>

extra: brincando com SMALI.

```
1  ..method public getNumber([II)Z
2
3      .registers 5
4
5      aget v0,v3,v4
6      const/4 v1,0x5
7
8      if-gt v0,v1, :cond_true
9
10     const/4 v2,0x0
11     return v2
12
13     :cond_true
14     const/4 v2, 0x1
15     return v2
16
17 ..end method
```

> <CONTEÚDO>

extra: código original em java

```
1  public function verifyNumber(int array[], int i) {  
2      int secret = array[i]  
3      if(secret > 5) {  
4          return true  
5      } else {  
6          return false  
7      }  
8  }
```

> <CONTEÚDO>

0x07: hands-on

- toolkit:
 - android virtual device
 - vs code
 - adb
 - apktool
 - uber apk signer
 - burp suite

OBRIGADO

